The Telolexic Method 2.0: A Universal Inverse-Recursive Audit Protocol (TAP) for Multimodal Generative Systems

Udimudi Naga Raju
Independent Researcher - Telolexic Audit Protocol (TAP)
Pixel9AI Project
July 2026

Correspondence: github.com/Pixel9AI/TAP

Abstract

Artificial intelligence systems-spanning Transformers, diffusion models, state-space architectures, and autonomous agents-predominantly operate via forward-chaining conditional probability: P(s_{t+1} | s_t). While computationally efficient for generation, this unidirectional design produces systematic integrity failures: hallucinations in large language models (LLMs), physics violations in synthetic imagery, and temporal drift in video. Existing mitigations (RLHF, Constitutional AI, GAN discriminators, retrieval augmentation) primarily assess quality or preference alignment, not causal grounding.

This paper presents Telolexic Method 2.0, formalized through the Telolexic Audit Protocol (TAP)-a model-agnostic governance layer that treats any output sequence as a state vector S = [s_0, ..., s_n], anchors the terminal state s_n as reference truth, and performs inverse-recursive auditing (s_n -> s_0) to verify that each state element possesses valid causal precursors.

We introduce four standardized application profiles: (1) Telolexic Pre-verified Transformer (TPT) for text safety, (2) Visual TAP for inverse-optical consistency, (3) Viloma-Video Protocol for temporal object permanence, and (4) Viloma Cryptography for terminal-dependent integrity. We report prototype implementations in Python (LLM supervisor) and Kotlin Multiplatform (on-device visual TAP), benchmark results against 2025-2026 verifier models, and a commercialization roadmap for enterprise AI governance.

Keywords- Telolexia, Vilomapatha, Telolexic Audit Protocol, inverse-recursive auditing, hallucination detection, temporal consistency, visual grounding, agentic supervision, AI safety, model-agnostic governance.

I. Introduction

A. The Forward-Generation Assumption

Modern AI pipelines mirror human chronological perception: initialization -> transformation -> output. LLMs decode left-to-right; video models predict frame t+1 from frame t; diffusion models iteratively denoise from noise toward an image. This shared forward vector optimizes for fluency and throughput.

Verification, however, is naturally teleological-oriented toward ends. When auditing a legal conclusion, a rendered frame, a cryptographic footer, or an agent's committed action, practitioners begin with the known terminal artifact and ask: what conditions were necessary for this to exist?

B. Telolexia and Vilomapatha

We propose Telolexia (Greek: telos, end + lexis, structure/logic)-the computational discipline of terminal-first verification. Its operational protocol, Vilomapatha (Sanskrit: viloma, reverse + patha, reading), implements "reading in reverse" as a formal audit algorithm applicable across modalities.

Unlike passive replay (debuggers) or abstract goal search (backward chaining in logic programming), Vilomapatha audits sequential state streams for causal completeness.

C. Contributions (Version 2.0)

This revision extends the original 2025 specification with:

1. Formal verdict taxonomy aligned with production audit systems
2. Reference implementations (TAP-Agent, kotlinMobile)
3. Integration guidance for 2026 on-device stacks (LiteRT, MediaPipe, Gemini Nano)
4. Dual-model verifier architectures (HalluGuard, NLI-DeBERTa)
5. Multi-object tracking extensions for Viloma-Video (ByteTrack, EdgeTAM)
6. Enterprise deployment and licensing framework

II. Related Work and Distinctions

Approach	Mechanism	TAP Distinction
Backward Chaining (P	Goal-driven rule infere	TAP audits concrete state sequ
Reverse Debugging (rr	Execution replay	TAP is active-flags logic gaps a
GAN Discriminators	Real vs. fake classifica	TAP checks causality, not plau
RLHF / Constitutional	Human preference sha	TAP is post-hoc, model-agnost
RAG	External retrieval grou	TAP verifies internal causal ch
HalluGuard / MiniChe	Claim-level hallucinati	TAP provides unifying protoco
NLI-based verification	Premise-hypothesis en	TAP's terminal-claim extractio

TAP is complementary, not competitive, with retrieval and classifier-based approaches. It specifies when and how to audit, not only which model performs classification.

III. Mathematical Formulation

A. State Vector Model

Let a generative process produce:

S = [s_0, s_1, ..., s_n]

where each s_t may represent a sentence, video frame descriptor, detected object set, cipher block, or agent action log.

Forward generation:

s_t = f_theta(s_{t-1}, c)

where theta are model parameters and c is conditioning context (prompt, prior frames, keys).

B. Verification Function

TAP defines verification V such that for valid sequences:

V(s_t) => \exists P subset of U_{i=0}^{t-1} s_i : Causes(P, s_t)

Failure modes:

- Continuity Violation: Required P missing => PhysicsViolation
- Exigenesis: Element x in s_t with no precursor => AnomalyFlagged
- Verified: All elements causally grounded => Verified

C. The Vilomapatha Loop

```
  Algorithm: TAP_AUDIT(S, anchor_index=n)
  1. ANCHOR: S_ref <- s_n
// Terminal truth
  2. FOR t FROM n DOWNTO 1:
  3.   required <- ExtractRequirements(s_t)
  4.   precursors <- U_{i=0}^{t-1} s_i
  5.   IF exists d in required : d not in
precursors:
  6.     RETURN PhysicsViolation(d, t)
  7.   IF exists x in s_t : x not in
precursors AND x not in S_ref:
  8.     RETURN AnomalyFlagged(x, t)
  9. RETURN Verified
```

D. Complexity

Per-audit complexity is O(n * |s|) for sequence length n and average state cardinality |s|. For streaming video, sliding-window audits reduce to O(k * |s|) for window k << n, enabling real-time edge deployment.

IV. System Architecture

A. TPT - Telolexic Pre-verified Transformer

Architecture:

```
User Prompt -> [Agent A: Generator LLM] ->
Buffered Response
v
                    [Agent B: Telolexist] <-
Terminal Claim Extraction
v
                              NLI / HalluGuard
Verdict
v
                         Release | Block |
Flag Response
```

2026 Recommended Stack:

- Agent A: qwen3:8b, gemma3:12b, or enterprise API
- Agent B: HalluGuard-4B, MiniCheck-7B, or deberta-v3-small-nli
- Optional: BGE-small embeddings for claim-context similarity

Implementation: TAP-Agent/telolexia_agent.py - Ollama-integrated supervisor with mock fallback.

B. Visual TAP - Inverse-Optical Grounding

For image I_{gen}:

1. Extract shadow and highlight regions
2. Compute inverse light vectors v_i

3. Test convergence: Var({v_i}) < epsilon
4. Reject if multi-source lighting implied without physical cause

Use cases: Synthetic media QA, advertising compliance, VFX pipeline gates.

C. Viloma-Video Protocol - Temporal Consistency

Frame-level formulation:

- Anchor frame F_T (terminal truth)
- Audit frame F_{T-1} for object set O
- Extended (2.0): Track IDs tau_i with embeddings e_i

Continuity predicates:

- Mass preservation: |O_T|_{tau} subset of Descendants(O_{T-1})
- Label stability: class(tau_i, T) ~ class(tau_i, T-1)
- Embedding drift: cos(e_T, e_{T-1}) > delta

2026 Recommended Stack:

- Detection: YOLO11n / EfficientDet-Lite (LiteRT .tflite)
- Tracking: ByteTrack, EdgeTAM
- Re-ID: MobileCLIP (quantized)

Implementation: kotlinMobile/shared/.../TelolexicAuditor.kt + CameraManager.android.kt (CameraX + ML Kit, upgradable to MediaPipe Tasks).

D. Viloma Cryptography

Terminal-dependent hash primitive:

H(S) = g(s_n (+) s_{n-1} (+) ... (+) s_0)

Retro-causal property: Footer-derived keys validate header integrity-supporting all-or-nothing stream security.

Implementation: TAP-Agent/telolexic.py - AES-CBC nesting with chi-square bias analysis.

E. Agentic Supervision

In multi-agent environments, TAP functions as a non-bypassable supervisor:

Role	Responsibility
Agent A	Generate code, text, plans
Agent B (Telolexist)	Inverse audit before commit/execute
TAP Gateway	Policy enforcement, audit logging

V. Experimental Results

A. Text Hallucination Detection

Setup: Supervisor Agent architecture, Vilomapatha audit on terminal claims.

Experiment	Model	Prompt	TAP Verdict
Future history	Gemma-3 (simul	"US history in 2(	FAIL - terminal claim unsupp
Cipher task	Gemma-3 (simul	Caesar encrypt "]	PASS - inverse decrypt restor

Broader benchmark (n=50, Llama-3 historical summaries): TAP identified logical discontinuities in 15% of responses rated "high quality" by surface fluency metrics (BERTScore).

2026 projection: Integrating HalluGuard-4B as Agent B is expected to improve balanced accuracy to 77-84% on RAGTruth-style benchmarks (per ACL 2026 findings).

B. Cryptographic Inverse Verification

AES-CBC two-level nesting with independent decryption audit:

- Forward: P -> C_1 -> C_2
- Inverse: C_2 -> C_1 -> P'
- Verdict: PASS iff P' = P
- Bias analysis: Chi-square on byte distributions confirms expected randomness properties

C. Visual TAP Prototype (Android)

On-device demo (kotlinMobile):

- Live object detection via ML Kit (upgradable to YOLO11n)
- Terminal anchor: snapshot of detected object labels
- Audit: set-difference continuity check
- Verdicts: Verified, AnomalyFlagged (new objects), PhysicsViolation (missing objects)

Limitation (acknowledged): v1 prototype uses label sets, not track IDs. v2 roadmap integrates ByteTrack + embeddings.

VI. 2026 Technology Integration Matrix

TAP Layer	2026 Technology	Deployment Target
Text verifier	HalluGuard-4B, MiniC	Server / edge LLM
Text generator	Qwen3, Gemma3, Llar	Ollama / cloud API
On-device vision	LiteRT 2.1, MediaPipe	Android (Pixel, Samsung)

On-device reasoning	Gemini Nano (AICore	Pixel 8 Pro+
Video tracking	EdgeTAM, ByteTrack	Mobile / edge GPU
Agent orchestration	TAP Supervisor Gatew	Enterprise middleware

VII. Commercial Applications

Industry	TAP Profile	Value Proposition
Cloud AI	TPT API	Hallucination governance as a service
Media & Entertainmen	Viloma-Video	AI continuity QA for film/TV/ad product
Device OEMs	On-device Visual TAP	Creator tools, camera continuity assista
Finance / Legal / Healt	TPT + audit logs	Regulatory-compliant copilots
Autonomous Agents	Supervisor Agent	Safe code/plan execution
Cybersecurity	Viloma Cryptography	Terminal-dependent stream integrity

Business model options: Open-core protocol (Apache 2.0 reference) + enterprise TAP Gateway (SaaS), OEM licensing, audit API metering.

VIII. Limitations and Future Work

1. Label-only visual prototype - Upgrade to tracked object identities and embeddings
2. Single-model audit in v1 Python agent - Migrate to dedicated verifier models
3. Physics heuristics - Visual TAP shadow analysis requires CV module integration
4. Formal proofs - Viloma-Hash cryptographic properties need peer-reviewed security analysis
5. Benchmark scale - Expand beyond 50-sample text evaluation to HaluEval, FEVER, SimpleQA
6. Latency - Characterize TAP overhead vs. generation time across modalities

IX. Conclusion

The Telolexic Method 2.0 reframes AI safety from "generate better" to "verify backward." By anchoring terminal truth and auditing causal precursors, TAP provides a unified governance protocol across text, vision, video, cryptography, and autonomous agents-without requiring retraining of foundation models.

As generative AI transitions from chat interfaces to autonomous production systems, inverse-recursive auditing is not optional infrastructure-it is the missing safety layer between creation and trust.

The reference implementations in this repository demonstrate that TAP is buildable today on commodity hardware. The research agenda is to harden, benchmark, and standardize it for industry adoption.

X. References

[1] Vaswani, A., et al. (2017). "Attention Is All You Need." NeurIPS.

[2] Naga Raju, U. (2026). "The Telolexic Manifesto 2.0: Inverse Narrative Deduction for Trustworthy AI." Pixel9AI Project.

[3] Naga Raju, U. (2026). "The Telolexic Method 2.0: TAP for Multimodal Generative Systems." Pixel9AI Project. (This document.)

[4] ACL Findings (2026). "HalluGuard: Evidence-Grounded Small Reasoning Models for RAG Hallucination Mitigation."

[5] Google AI Edge (2026). "LiteRT for Android - CompiledModel API." developers.google.com/edge/litert

[6] Huang, Y., et al. (2025). "EdgeTAM: On-Device Track Anything Model." CVPR 2025.

[7] Farquhar, S., et al. (2024). "Detecting Hallucinations in Large Language Models Using Semantic Entropy." Nature.

[8] EU Artificial Intelligence Act (2024/2026). Regulatory framework for high-risk AI systems.

[9] Vedic Linguistics. "Viloma Paatha: Reverse Recitation Structure." Traditional pedagogical reference.

[10] NIST SP 800-series. Cryptographic standards (comparative analysis for Viloma-Hash).

Appendix A: Repository Structure

```
Pixel9AIMain/
|-- docs/
|   |-- Telolexic_Manifesto_2.0.md
|   |-- Telolexic_Method_Whitepaper_2.0.md
|   +-- PITCH_ONE_PAGER.md
|-- TAP-Agent/
|   |-- telolexia_agent.py      # Text TAP
supervisor
|   |-- telolexic.py            # Crypto TAP
module
```

```
|   +-- requirements.txt
+-- kotlinMobile/
    +-- shared/.../TelolexicAuditor.kt  #
Visual TAP core
```

Appendix B: Verdict Schema (JSON)

```
{
  "protocol": "TAP-2.0",
  "verdict": "Verified | AnomalyFlagged |
PhysicsViolation",
  "anchor_state": "S_n",
  "audit_direction": "inverse",
  "timestamp": "ISO-8601",
  "details": "Human-readable causal
explanation"
}
```
